

schengen: Constraint-Guided Schematic Generation for Analog Circuits via Large Language Models

Anonymous Authors
Anonymous Institution(s)

Abstract—Schematic generation is essential in analog synthesis and topology design, enabling design understanding, expert feedback, and documentation. Recent approaches based on reinforcement learning (RL) and large language models (LLMs) formulate this task as optimization or translation from SPICE netlists to schematics. However, they are often computationally expensive, limited to small circuits, or lack controllability. Constraint-driven placement is long established in analog automation, but its constraints come from expert knowledge or automated subcircuit recognition, both unavailable precisely on the novel and synthesized topologies that recognizers do not cover well. We introduce *schengen*, which applies constraint-driven placement to schematic generation and adds a large language model as a third constraint source for exactly that regime, with an interactive GUI for fine-grained control. Our contribution is to quantify what this source adds. Across 15 benchmark circuits (14–48 devices) and 10 random seeds each, LLM-generated constraints reduce bends, net crossings, and HPWL by 21%, 27%, and 17% over a classical constraint-driven configuration. All source code, benchmarks, and generated schematics are available at: <https://anonymous.4open.science/r/schengen>.

Index Terms—Schematic generation, Large language models, Analog circuits

I. INTRODUCTION

Analog topology discovery and synthesis [1–5] have advanced rapidly with automation and machine learning, enabling large-scale generation of SPICE netlists. However, these netlists are low-level and difficult to interpret, limiting their usefulness for design analysis, debugging, and expert feedback. Converting SPICE netlists into human-readable schematics is therefore a critical step for design understanding, validation, and documentation. Recent netlist-to-schematic approaches include LLM-based methods [6], RL [7, 8], and conventional optimization [9]. While promising, they suffer from high computational cost (e.g., RL training), limited scalability beyond small circuits, and a lack of fine-grained controllability that would let designers guide or refine results. To address these limitations, we adopt the *constraint-driven* methodology that is long established in analog layout automation [10, 11] and apply it to schematic generation. Instead of generating schematics directly, we define a set of constraint primitives that govern component placement and routing on a grid-based canvas. The resulting constraint system is solved using a SAT/CP (i.e., constraint programming) solver integrated with a custom placement and routing engine. Constraints in that methodology conventionally come from expert knowledge or from automated subcircuit recognition, and both are unavailable precisely where they are most needed: on novel or synthesized topologies that recognizers do not cover. Our central claim is that a large language model is

an effective alternative source, supplying structural placement constraints for circuits where the two classical aforementioned sources are limited or unavailable. Based on this formulation, we develop *schengen*, an automated tool for schematic generation from SPICE netlists using constraint programming, which also supports interactive refinement so that designers can improve readability and structural quality. Our main contributions are as follows:

- We reformulate netlist-to-schematic conversion as a constraint generation problem and introduce a set of primitives for constructing human-readable schematics. We show that LLM-generated constraints significantly improve schematic quality, and that integrating LLMs as a constraint source is more effective than end-to-end schematic generation.
- We present *schengen*, an automated framework combining constraint solving with placement and routing, offering an interactive graphic user interface (GUI), to better support human-in-the-loop workflow that improves interpretability, controllability, and schematic quality.
- We demonstrate that incorporating LLM-generated constraints enhance the quality of generated schematics. When Opus 4.8 is used for constraint generation, across 15 circuits, it helps to reduce crossings by 27%, bends by 21% and HPWL by 17%, and overall variability between different runs. Similar improvement trend can be observed with GPT 5.4, showing the robustness and consistency performance across different language models.

II. RELATED WORK

Early research in schematic generation relied on heuristic optimization and staged placement for digital circuits [12–15], while analog designs mainly adopted reinforcement learning [7]. Recent LLM-based approaches include Schemato [6] for netlist-to-LTSpice conversion, frameworks generating SKiDL code and PCB layouts [16, 17], and PCBSchemaGen [18] using knowledge graphs. Work on constraint-driven automation integrates expert knowledge [10] and uses machine learning to generate structural constraints [11, 19] that guide topology and layout generation. EDA companies have patented device clustering and rule-based group placement [20–22], yet commercial tools still fail to produce aesthetic schematics for many analog circuits. Unlike RL methods requiring extensive training [7, 8] or direct LLM approaches that struggle with large-scale circuits [6], *schengen* formulates schematic generation as discrete constraint satisfaction, combining grid-based placement with LLM-driven constraint generation for scalable, training-free schematic generation.

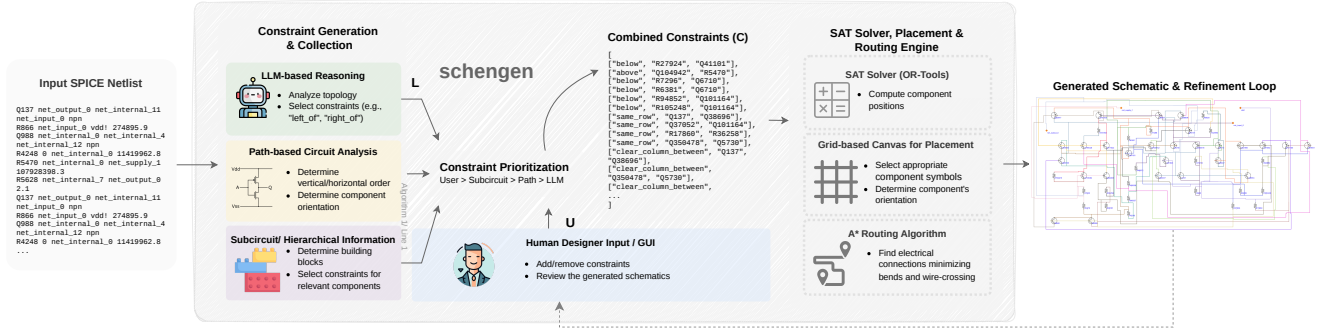


Fig. 1. Overview of the schengen framework (Sec. III).

III. THE PROPOSED METHOD

A. Method Overview

Fig. 1 illustrates the proposed schengen framework, which supports common analog components including MOSFET/BJT transistors, capacitors, and resistors. To reduce the continuous design space, we adopt a grid-based canvas, where each cell is indexed from (0,0) at the bottom-left. All components are initially unassigned to grid locations. Placement is formulated as a constraint generation problem, where constraints derived from LLM reasoning, heuristic rules (e.g., path-based analysis), and user input define relative positions and orientations (e.g., alignment, adjacency). Constraint primitives are detailed in Sec. III-B. The overall placement procedure is given as pseudocode in Algo. 1. It proceeds in two stages: the LLM-generated batch is admitted against the base B of user- and path-derived constraints by bisection over feasibility checks (line 13), and each recognized subcircuit group is then tested against the admitted set, resolving conflicts in favour of the recognizer (line 15). The resulting constraint set is solved once under the routing-aware objective (lines 21–22). A constraint programming solver (e.g., OR-Tools [23] as in our implementation) then computes feasible placements satisfying the combined constraint set while optimizing three routing-aware objectives: compactness (maximum occupied rows), Half-Perimeter Wire Length (HPWL), and estimated routing cost (total Manhattan distance). Finally, given the placement, an A*-based routing algorithm establishes wire connections, producing the complete schematic.

B. Constraint Set Primitives

schengen provides 16 constraint primitives, all following the uniform format [$\langle \text{constraint} \rangle$, ($\langle \text{value} \rangle$), $\langle \text{target} \rangle$, $\langle \text{references} \rangle$], where $\langle \text{constraint} \rangle$ is the spatial or structural relation, $\langle \text{target} \rangle$ the component constrained, $\langle \text{references} \rangle$ the components it depends on, and the optional $\langle \text{value} \rangle$ an additional parameter. For example, [left_of , $m1$, $m2$] enforces that the column index of $m1$ is smaller than that of $m2$. The complete list with the total of 16 different constraints is provided in the supplementary material [24], and each is translated into solver-compatible form (for us OR-Tools’ built-in constraints).

Algorithm 1 Constraint-based Placement in schengen

Require: Netlist N , LLM constraints L , user constraints U , CP model m
Ensure: Placement and orientation solution S

- 1: $B \leftarrow U \cup \text{ANALYZECIRCUIT}(N)$
- 2: $G \leftarrow \text{RECOGNIZEBUILDINGBLOCKS}(N)$
- 3: **function** FEASIBLE(C) ▷ Yes/no check
- 4: **return** $m.\text{SOLVE}(\text{TRANSLATE}(C)) \neq \text{infeasible}$
- 5: **end function**
- 6: **function** ADMIT(Q, C) ▷ Batch bisection
- 7: **if** $Q = \emptyset$ **or** FEASIBLE($C \cup Q$) **then return** Q
- 8: **else if** $|Q| = 1$ **then return** $\emptyset$
- 9: **end if**
- 10: $Q_1, Q_2 \leftarrow \text{SPLITINHALF}(Q)$; $A_1 \leftarrow \text{ADMIT}(Q_1, C)$
- 11: **return** $A_1 \cup \text{ADMIT}(Q_2, C \cup A_1)$
- 12: **end function**
- 13: $A \leftarrow \text{ADMIT}(L, B)$ ▷ Stage 1: admit the LLM batch against the base
- 14: $\hat{G} \leftarrow \emptyset$
- 15: **for all** $g \in G$ **do** ▷ Stage 2: subcircuit de-confliction
- 16: **if** FEASIBLE($B \cup A \cup \hat{G} \cup \{g\}$) **then** $\hat{G} \leftarrow \hat{G} \cup \{g\}$
- 17: **else if** FEASIBLE($B \cup \hat{G} \cup \{g\}$) **then** ▷ Recognizer wins: drop the LLM relations g displaces
- 18: $\hat{G} \leftarrow \hat{G} \cup \{g\}$; $A \leftarrow \text{ADMIT}(A, B \cup \hat{G})$
- 19: **end if** ▷ Otherwise g is unplaceable against the base: dropped
- 20: **end for**
- 21: $m.\text{SETCONSTRAINTS}(\text{TRANSLATE}(B \cup A \cup \hat{G}))$
- 22: **return** $m.\text{MINIMIZE}(\text{ROUTINGCOST}(N) + H)$ ▷ H : schematic size + HPWL

schengen additionally enforces implicit default constraints that capture widely adopted design conventions: (D1) preventing component overlap, (D2) enforcing minimum spacing between components (e.g., at least two rows and columns apart), (D3) avoiding placement near the canvas boundaries, and (D4) positioning VDD terminals at the top and GND terminals at the bottom (if present). These are applied by default without over-constraining the solution space. No predefined constraints are imposed on input/output terminals. Their placement is determined by the constraint sources described in the following subsections.

C. LLM-generated Constraints for Schematic Generation

Previous work like [25, 26] shows that LLMs can analyze SPICE netlists and identify functional building blocks. Inspired by that, we leverage LLMs to generate the placement constraint set. The input is the SPICE netlist and the constraint set primitives, optionally with free-form subcircuit or partitioning descriptions to strengthen the model’s reasoning. The model then selects suitable constraints from the circuit structure

and functionality. Incorporating LLM-generated constraints improve the overall topology structure coverage, as rule-based recognizers may lack heuristics [3], while learning-based methods lack training data [27] for generating proper constraints for unseen topologies.

Constraint Admission The resulting LLM-generated constraints complement those from the other constraint sources (which will be provided in the subsequent subsections). They are incorporated iteratively, each admitted only if it does not conflict with higher-priority constraints (ADMIT, Algorithm 1 lines 6–11, invoked in line 13 and again during de-confliction in line 15), so that LLM suggestions enhance rather than degrade the placement. LLM constraints are first admitted against the user- and path-based constraints (if available): the batch of LLM-generated constraints is tested for joint feasibility and, when that fails, bisected so that only the offending constraints are dropped. Each recognized subcircuit group is then tested against the admitted set. We further restrict the LLM to select among provided primitive constraints to avoid LLMs come up with unknown constraints, and also explicitly prompt the LLM to follow an *iterative strictness* strategy: begin with constraints that preserve placement flexibility (e.g., `right_of`) and progressively introduce stricter ones that sharply reduce the feasible space (e.g., `same_row`, which enforces exact alignment). This supports a progression from relaxed to strict constraints in the admission procedure, enabling an incremental refinement process.

D. Enhancing Schematic Quality with Additional Constraint Sources

In addition to LLM-generated constraints, `schengen` also aggregates constraints from other additional sources (i.e., subcircuit/partition information, circuit analysis, and user-defined), which are to be always correct by nature and are incorporated prior to LLM-generated constraints, aiming to reduce the solution space and improve consistency.

1) *Subcircuit and Partitioning Information*: Subcircuit and partitioning information provide strong structural priors for placement, obtainable via heuristics, template matching, or ML-based methods. For operational amplifiers we use ACST [28, 29] for accurate rule-based extraction. For general circuits, ML approaches such as GANA [27] and GENIE-ASI [25] can be used, although false positives may degrade placement quality. Furthermore, `schengen` maintains a library of common subcircuits (e.g., differential pairs, current mirrors, inverters) with predefined placement constraints that encode domain knowledge such as alignment and symmetry ([24]). Once a subcircuit is identified, its constraints are applied automatically, ensuring that recognized structures are placed in a human-readable manner.

2) *Path-based Circuit Analysis (Traceability Principles)*: We further derive placement constraints from current flow paths (VDD to GND) and common schematic conventions [7], since components sharing electrical connections are typically aligned in human-designed schematics: **P1**. Same-type components with drain–source connections are placed in the same column, **P2**. Different-type components with drain–drain or source–source

connections are placed close in the same column, **P3**. No components are placed between connected components (P1, P2), **P4**. Same-type components sharing gate connections are aligned in the same row, **P5**. Components connected to VDD (GND) are placed in the upper (lower) region, **P6**. Input/output ports are placed near connected components. These constraints improve traceability and readability. All are enabled by default, but users can selectively apply them via the GUI (Sec. V).

3) *User-defined Constraints*: Designers supply constraints interactively through the GUI (Sec. V), in the same format as LLM-generated ones. This allows designers to define, modify, and remove constraints based on their own preferences and other requirements.

E. Constraint Prioritization

When multiple constraints are imposed, conflicts may arise. `schengen` therefore applies constraints in the high-to-low priority order: (1) user-defined, path-based, subcircuit-based (i.e., share the same priority), and (2) LLM-generated. Higher-priority constraints are enforced first and cannot be overwritten. If any conflicts arise between them (e.g., due to a conflicting user-provided constraint), no valid placement solution is obtained. LLM-generated constraints are incorporated only if they do not violate them. This preserves critical structural knowledge while allowing LLMs to resolve placement ambiguities, which is especially useful when subcircuit recognition fails, as in novel synthesized topologies.

F. Determination of Component Orientation

Component orientation affects both readability and routing efficiency. For simplicity, in this initial version of `schengen`, each component has two possible orientations: for transistors, gate/base terminal facing right (0) or left (1); for two-terminal devices, positive terminal facing up (0) or down (1). Initial orientations are assigned using three strategies. (i.) *Structural rules*: recognized subcircuits directly define orientations (e.g., symmetric devices in a current mirror are assigned opposite orientations, [24]). (ii.) *Connectivity-based heuristic*: remaining components are oriented to face the side with more connections, reducing routing complexity. (iii.) *Gate-net alignment*: once placement is fixed, each transistor not already determined by (i.) is oriented so that its gate terminal faces the side on which its gate net actually lies. Because the gate net is frequently the longest connection a device participates in, and gate rails are drawn as shared horizontal wires, this shortens wiring and avoids routing a gate connection around the device body. Additionally, users can explicitly specify orientations through the GUI provided (e.g., {"M1": 0, "M2": 1, "R3": 1}). User-defined orientations have the highest priority and overwrite automatic assignments, enabling fine-grained control after inspection.

IV. WIRE ROUTING WITH A* SHORTEST-PATH ALGORITHM

After determining component placement and orientation, `schengen` routes all nets while preserving readability and efficiency. For each net, connected components are identified

from the SPICE netlist and pin positions are inferred from component locations, orientations, and symbol templates. Since each component occupies a single grid cell, pins lie at the midpoints of cell edges, making routing unambiguous. Interconnections are then generated using A* search, minimizing Manhattan distance, congestion cost, wire length and bends. Unlike physical layout, wire crossings are unavoidable in schematics [8], so `schengen` permits controlled crossings under a small penalty, which often yields more compact and visually coherent layouts.

Routing order significantly affects quality, so nets with more connections are routed first, allowing complex nets to establish primary routes early. Routing proceeds incrementally: starting from one pin, each new pin connects to the nearest routed node, forming a trunk-and-branch (Steiner-like) structure under the A* cost described above. In addition, we also introduce two refinements to target the cases where this scheme performs poorly: (i.) Human schematics draw gate-to-gate wires straight through aligned devices, so narrow regions between aligned components are marked routable during search, and (ii.) High-fanout nets such as supply and bias nets, which connect a large fraction of the devices, commit to a single straight trunk spanning the net’s extent and connect each pin by a short perpendicular wire. This reproduces the power-rail convention of hand-drawn schematics and removes the dependence on pin visiting order that makes incremental routing unstable.

Finally, a guarded cleanup pass deduplicates coincident segments, removes tails that terminate at no pin, and merges collinear segments to reduce bends. It never removes a segment that is the sole connection to a pin, so it cannot disconnect a routed net. In addition, `schengen` also applies the drawing conventions analog designers expect: crossings between unrelated nets are drawn as hops so that a crossing is visually distinguishable from a junction, supply nets are drawn at the top and ground at the bottom, and gate rails are drawn as shared horizontal wires. These conventions do not change placement or connectivity, but they are what the human-readability criterion of Sec. VI ultimately measures.

V. INTERACTIVE REFINEMENT WITH THE `SCHENGEN` GUI

Automatic generation rarely captures every preference a designer has for a schematic, so `schengen` pairs the solver with a web-based GUI built around a simple principle: the designer can further refine the schematic to match their unique preferences, with the tool proposing a complete, valid drawing and the designer steering it where automation falls short. The GUI exposes four levers. **Path-based constraint selection:** the traceability principles P1–P6 (Sec. III) are enabled by default but can be toggled individually, which matters on unconventional topologies where a subset works better than the full set (Sec. VI). **User-defined constraints:** designers can add, modify, and remove constraints in the same primitive vocabulary the LLM uses; they enter the solver at the highest priority (Sec. III) and thus override any automatically derived relation. **Visualization:** every change re-solves and re-renders the schematic immediately, and active constraints are visualized on the canvas, so the effect of each edit is visible before the next one is made. **Component orientation:** device orientations chosen by the automatic rules

can be overridden per component. Screenshots of the GUI and the full constraint list are provided in [24].

It is worth mentioning that none of the experiments in Sec. VI uses the GUI: every result is produced in a single, fully automatic run per configuration, and no user-defined constraints are provided. Thus, no configuration benefits from interactive or subjective refinement, and thus allows a fair comparison.

VI. IMPLEMENTATION & EVALUATION

A. Tool Implementation

`schengen` is implemented in Python using OR-Tools [23] as the constraint solver and Flask [31] for the web application. The implementation is publicly available at [32], and the GUI, which loads netlists and settings and adds, removes or selects constraints interactively, is shown in [24]. It takes a flat SPICE netlist plus optional subcircuit information, LLM-generated constraints, and user-defined constraints. On an AMD Ryzen Threadripper PRO 5955WX (16 cores, 126 GB RAM, Ubuntu 20.04), the median end-to-end runtime over the 15 benchmark circuits is 14.1 s without LLM-generated constraints, 20.6 s with Opus 4.8 constraints and 34.0 s with the more numerous GPT-5.4 constraints. The gap is entirely dominated by constraint admission, a median 71% of runtime and scaling with the number of proposed constraints, while placement and routing together stay at a few seconds even on the largest circuit ([24]). Constraint admission is thus the main runtime bottleneck when applying `schengen` to industrial-scale circuits, but it is also the most tractable, because its feasibility checks are independent although currently run sequentially. Constraint generation is negligible by comparison: we require one LLM call per circuit with overall roughly 68k input and 6–11k output tokens for the *entire* benchmark, with no finetuning. Generated schematics can be exported in `.png` and `.asc` (LTSpice) formats, and rendered from the command line for batch schematic generation.

B. Evaluation

Benchmark Circuits: We separate qualitative from quantitative evaluation. For *qualitative* comparison we use three circuits having an external reference schematic: a differential operational amplifier [30], a benchmark from [7] for which both a prior method’s and a commercial tool’s schematics are published, and a circuit from our in-house synthesis engine. For the first two, netlists are derived from the reference schematics and subcircuits extracted using ACST [29]. For *quantitative* evaluation we use 15 circuits (with 14–48 devices): the three above and twelve further operational amplifiers spanning single-, two- and three-stage topologies, single-ended and fully-differential, including nested-Miller-compensated (NMC) and reversed-NMC variants. Subcircuit information is available for only 5 of the 15, reflecting the practical reality that recognition does not cover novel or synthesized topologies. No user-defined constraints are applied to any circuit to avoid any subjective comparison, and the two LLM configurations use the prompt template of Sec. III-C unchanged, differing only in which language model is queried.

Evaluation Metrics: Schemato [6] treats generation as translation and evaluates via *Graph Edit Distance* and *Compilation Success Rate*. `schengen` formulates it as placement

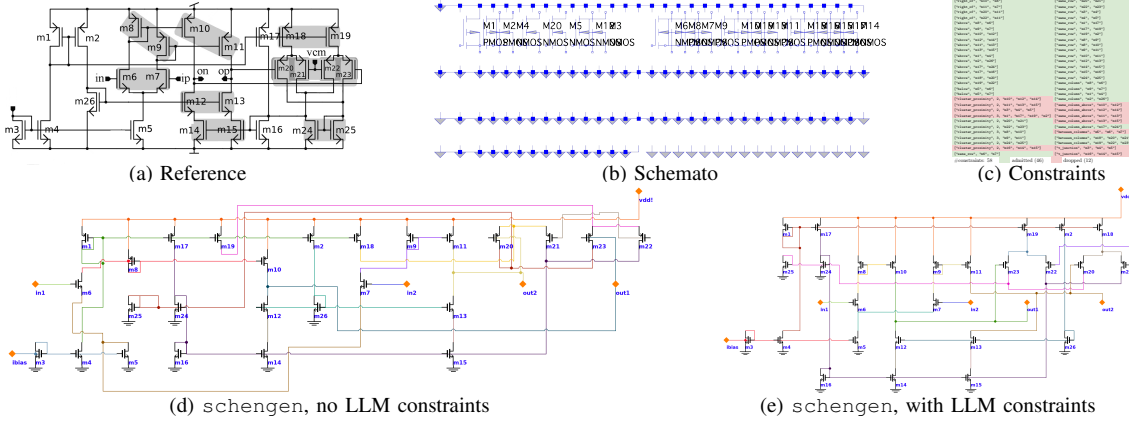


Fig. 2. (Example 1) Comparison of reference [30], Schemato, and schengen schematics.

and routing, ensuring structural and functional correctness by construction, which makes such metrics vacuous here. We therefore adopt the aesthetic metrics introduced for this purpose by AEM-PCB [8] (wire bends, net crossings, HPWL, and alignment neatness) rather than defining our own. The first three are quantitative, while alignment neatness is, in *schengen*, largely enforced wherever subcircuit templates are available. Given the lack of standardized metrics and limited access to baselines [7], we first perform *qualitative visual comparisons* against reference schematics, assessing (i) visibility of functional blocks and (ii) compactness of placement, and then evaluate routing quality quantitatively across configurations within *schengen*. All schematics are publicly available at [32].

1) Example 1: Standard differential operational amplifier circuit: We compare *schengen* with and without LLM constraints against Schemato, using Opus 4.8. As shown in Fig. 2b, Schemato fails to generate a valid schematic: the produced .asc file is syntactically incorrect, and after minimal correction using GPT-5.4 it becomes loadable in LTSpice but remains unstructured, with misplaced components and disconnected wires. It is worth mentioning that the circuit in this example lies outside the size range Schemato was trained on, so the comparison established here should be interpreted with caution. *schengen* (Fig. 2d, 2e) exposes functional blocks: the differential pair and Gilbert cell (m20–m23) are identifiable, and conventions such as alignment of drain–drain connections are preserved. LLM-generated constraints tighten two structures that the classical sources leave loose: the input pair m6/m7 closes from twelve columns to four, and m18/m19 move to within one column of the devices they drive, so the Gilbert cell reads as a single block. As shown in Fig. 2c, 46 of the 58 proposed constraints (~79%) are retained after admission, in line with the 80% this model retains on average. Which constraints survive is informative: all twelve dropped come from the stricter primitives, though strictness alone does not decide it, since other constraints of those same kinds survive. What is dropped is what *conflicts*, with another LLM constraint or with a higher-priority subcircuit group. Finally, *schengen* is training-free and model-agnostic, enabling seamless integration of future improved LLMs without retraining.

2) Example 2: Comparison with Prior Work and a Commercial Tool: We evaluate *schengen* on the circuit from [7] to enable direct comparison. Since the implementation of Hsu et al. [7] is not publicly available and the commercial tool in [22] is inaccessible, we use the published schematics as references (Fig. 3a, 3b). Both are included for qualitative comparison. As Schemato again produces invalid results (similar to Example 1), we omit it for brevity. Similar to Example 1, for this circuit, *schengen* employs a combined constraint set constructed from LLM-generated, path-based, and subcircuit-based constraints. Compared to the reference schematics, *schengen* more clearly exposes functional structures. For instance, in [7], transistors m17 and m18 are placed far apart, making the differential pair less apparent. In contrast, *schengen* (Fig. 3c) places them in close proximity with consistent alignment, enabling immediate recognition. Moreover, in the reference schematics, transistors sharing gate connections are not consistently aligned, reducing readability and complicating signal tracing. *schengen* enforces row alignment for such devices, resulting in clearer connectivity and a cleaner layout. In terms of compactness, *schengen* and the method from [7] produce schematics with comparable height (maximum stacked transistors), while the commercial tool yields a significantly taller layout. Additionally, *schengen* generates the schematic in under two minutes, whereas RL-based approaches typically require substantial runtime. This difference is important in practice, where fast iteration is critical.

3) Example 3: Unconventional Synthesized Analog Circuits: We further evaluate *schengen* on a synthesized circuit implementing a cubic function, i.e., the output voltage is the cube the input voltage, which lacks standard building blocks and is therefore a case the classical constraint sources handle worst: no subcircuit information is provided and path-based analysis applies only partially. Even so, enforcing the subset **P1**, **P3**, **P6** improves structural regularity, and LLM-generated constraints further group related components and reduce routing density ([24]). Overall, *schengen* produces compact and structured schematics even for unconventional topologies with minimal constraints. The ability to incorporate additional constraints and export results in .asc format also enables efficient manual refinement and downstream design optimization.

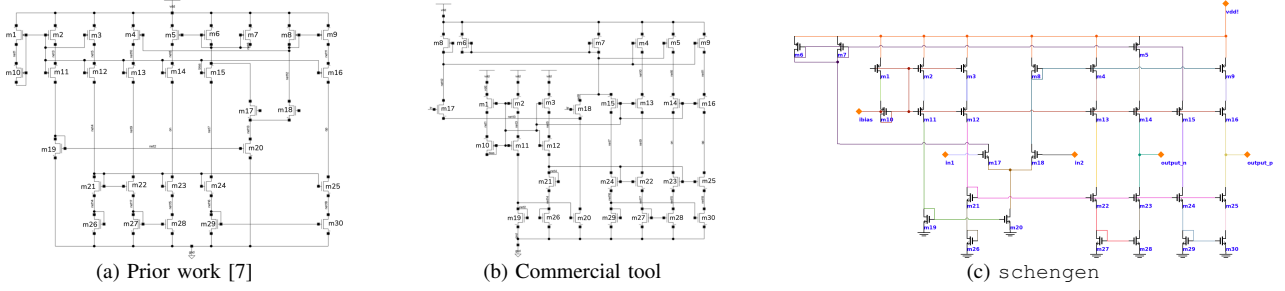


Fig. 3. (Example 2) Schematic comparison: prior work [7], a commercial tool, and *schengen*.

TABLE I
EFFECT OF LLM-GENERATED PLACEMENT CONSTRAINTS ON ROUTING QUALITY (15 CIRCUITS, 10 SEEDS, LOWER IS BETTER).

Configuration	Routing quality				Reprod. (μ_{seed})		Cost	
	#crossings	#bends	HPWL	Routing cost (10^3)	#crossings	HPWL	Completed	Runtime
w/o LLM constraints	661 $\pm$ 44	802 $\pm$ 36	3322 $\pm$ 121	500 $\pm$ 23	8.3	24.9	149/150*	14.1 s
w/ LLM (Opus 4.8)	486$\pm$30 (-27%)	630$\pm$27 (-21%)	2772$\pm$49 (-17%)	343$\pm$12 (-31%)	4.3	7.0	150/150	20.6 s
w/ LLM (GPT-5.4)	521 $\pm$ 34 (-21%)	664 $\pm$ 35 (-17%)	2948 $\pm$ 72 (-11%)	360 $\pm$ 15 (-28%)	5.6	10.1	150/150	34.0 s

*One w/o-LLM run (1/150) failed as the CP-SAT solver did not find a feasible solution within the 10-second limit.

4) *Visual Aesthetics Comparison*: We use routing quality metrics to compare the visual aesthetics of schematics generated by different constraint configurations, as a smaller routing cost corresponds to better placement of components that belong together. Table I summarizes routing quality without LLM-generated constraints and with constraints generated by two independent language models, Opus 4.8 and GPT-5.4, using identical prompts. Each circuit is run with 10 random seeds to account for solver randomness. Each cell is the sum of the per-circuit means over all 15 circuits, and percentages are relative to the configuration without LLM constraints. The $\pm$ on a sum says how much the benchmark total varies. The μ_{seed} column answers a different question and is defined in Sec. VII. Bends and crossings are counted on the routed topology as the router produces it. Our experiment results show that LLM-generated constraints reduce all four metrics. With Opus 4.8, net crossings fall by 27%, wire bends by 21%, HPWL by 17%, and routing cost by 31%. With GPT-5.4 the corresponding reductions are 21%, 17%, 11% and 28%. The observed results demonstrate that *schengen* benefits from LLM reasoning. LLM-generated constraints tends to place related components together and others slightly farther apart, trading wirelength for the routing space that removes bends and crossings. Between two language models, Opus 4.8 beats GPT-5.4 on all four metrics and is more reproducible, GPT-5.4 proposes 1166 constraints against 659 for the same circuit, of which a smaller fraction survives admission (75% versus 80%), so it takes $1.7\times$ as long for slightly worse schematics (see [24] for per circuit details).

VII. DISCUSSION

Performance Gains With and Without Subcircuit Information In our experiments, subcircuit information is available for only 5 of the 15 circuits. To measure the performance improvement with and without subcircuit information, we average the per-circuit crossing reductions (using the same runs

as in Table I) and observe that Opus 4.8 improves the ten circuits without recognition by 40% on average, compared with 15% for the five circuits where subcircuit information is provided. This indicates that LLM-generated constraints can complement constraints obtained from subcircuit information and further enhance the overall quality of the generated schematics.

Constraint Conflicts On average, constraint admission retains 86% of proposed constraints for Opus 4.8 and 81% for GPT-5.4. The lowest retention is 75% and 48% respectively.

Seed-to-Seed Reproducibility Beyond routing quality, LLM-generated constraints change how much a single run can be trusted. Because the solver chooses arbitrarily among the equally good placements that the constraint set leaves undetermined, an unconstrained problem yields visibly different schematics on different seeds. The μ_{seed} column of Table I measures that freedom: for each circuit we take the standard deviation of its crossing count over the 10 seeds, and μ_{seed} is the mean of those 15 values. LLM-generated constraints shrink this value: the crossing count varies by 8.3 without them and by 4.3 with Opus 4.8 constraints, and HPWL shows the same trend: 24.9 to 7.0, and four of the 15 circuits become fully deterministic across all ten seeds, against two without. This is important for a tool used iteratively, as a designer who adjusts one constraint and regenerates should see a schematic that differs because of the edit, not due to the randomness in the solver itself.

VIII. CONCLUSION AND OUTLOOK

We presented *schengen*, which applies constraint-driven placement to schematic generation and adds the LLM as a constraint source for the topologies rule-based recognizers do not cover. Across 15 circuits and 10 seeds it reduces crossings by 27%, bends by 21% and HPWL by 17% over a classical constraint-driven configuration. Future work will focus on improving scalability and incorporating automated feedback mechanisms. In particular, integrating agentic pipelines with vision-language models may enable iterative visual evaluation and constraint refinement.

REFERENCES

- [1] Haoyi Zhang et al. “Analogxpert: Automating analog topology synthesis by incorporating circuit design expertise into large language models”. In: *2025 International Symposium of Electronics Design Automation (ISED)*. IEEE. 2025, pp. 772–777.
- [2] Yuxuan Hou et al. “Cktgen: Specification-conditioned analog circuit generation”. In: *arXiv e-prints* (2024), arXiv:2410.
- [3] Inga Abel and Helmut Graeb. “FUBOCO: Structure synthesis of basic op-amps by FUnctional BLock COMposition”. In: *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 27.6 (2022), pp. 1–27.
- [4] Stefan Uhlich et al. “GraCo—A Graph Composer for Integrated Circuits”. In: *arXiv preprint arXiv:2411.13890* (2024).
- [5] Stefan Uhlich et al. “Spicing up Genetic Netlist Generation with LLMs”. In: *arXiv preprint arXiv:2608.23317* (2026).
- [6] Ryoga Matsuo et al. “Schemato—An LLM for netlist-to-schematic conversion”. In: *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*. IEEE. 2025, pp. 1–7.
- [7] Hung-Yun Hsu and Mark Po-Hung Lin. “Automatic analog schematic diagram generation based on building block classification and reinforcement learning”. In: *Proceedings of the 2022 ACM/IEEE Workshop on Machine Learning for CAD*. 2022, pp. 43–48.
- [8] Jie Yang et al. “AEM-PCB reverser: circuit schematic generation in PCB reverse engineering using reinforcement learning based on aesthetic evaluation metric”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43.5 (2023), pp. 1608–1612.
- [9] Bogdan G Arsintescu. “A method for analog circuits visualization”. In: *Proceedings international conference on computer design. VLSI in computers and processors*. IEEE. 1996, pp. 454–459.
- [10] Göran Jerke and Jens Lienig. “Constraint-driven design: The next step towards analog design automation”. In: *Proceedings of the 2009 international symposium on Physical design*. 2009, pp. 75–82.
- [11] Oliver Mitea et al. “Automated constraint-driven topology synthesis for analog circuits”. In: *2011 Design, Automation & Test in Europe*. IEEE. 2011, pp. 1–4.
- [12] B Naveen and KS Raghunathan. “An automatic netlist-to-schematic generator”. In: *IEEE Design & Test of Computers* 10.1 (2002), pp. 36–41.
- [13] Stephen T Frezza and Steven P Levitan. “SPAR: a schematic place and route system”. In: *IEEE transactions on computer-aided design of integrated circuits and systems* 12.7 (1993), pp. 956–973.
- [14] Nam-Hoon Kim et al. “RightTopologizer: an efficient schematic generator for multi-level optimization”. In: *Proceedings of 13th Annual IEEE International ASIC/SOC Conference (Cat. No. 00TH8541)*. IEEE. 2000, pp. 387–391.
- [15] Bikram Garg, Amarpal Singh, and Ashish Agrawal. “Efficient undo-redo with incremental netlist to schematic generation”. In: *2010 53rd IEEE International Midwest Symposium on Circuits and Systems*. IEEE. 2010, pp. 789–792.
- [16] Khandakar Shakib Al Hasan et al. “CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural Language Prompts”. In: *arXiv preprint arXiv:2601.04505* (2026).
- [17] Hongyuan Du, Xiaozhi Du, and Jinhua Yang. “ASGF4EPD: An Automated Schematic Generation Framework for Electronic Products Design”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2026).
- [18] Huanghao Zou et al. “PCBSchemaGen: Constraint-Guided Schematic Design via LLM for Printed Circuit Boards (PCB)”. In: *arXiv preprint arXiv:2602.00510* (2026).
- [19] Prasanth Mangalagiri et al. “CDLS: Constraint driven generative AI framework for analog layout synthesis”. In: *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 2024, pp. 1–6.
- [20] Balvinder Singh et al. *System and method for circuit schematic generation*. US Patent 7,917,877, 2011. 2011.
- [21] Amarpal Singh Bikram Garg Rajeev Sehgal. *Schematic generation visualization aid for netlists comprising analog circuits*. US Patent 8,504,953, 2013. 2013.
- [22] Tian-Hau Tsai et al. *Rule-based schematic diagram generator*. US Patent 7,386,823, June 2008.
- [23] Google. *OR-Tools*. 2026. URL: <https://developers.google.com/optimization> (visited on 01/30/2026).
- [24] *Supplementary material for schengen*. URL: <https://anonymous.4open.science/r/schengen/paper/supplementary.pdf>.
- [25] Phuoc Pham et al. “GENIE-ASI: Generative Instruction and Executable Code for Analog Subcircuit Identification”. In: *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*. IEEE. 2025, pp. 1–21.
- [26] Andrei Dăescu et al. “Evaluating the Impact of Circuit Representation on LLM-Based Functional Block Recognition in Analogue Circuits”. In: *Electronics Letters* 62.1 (2026), e70512.
- [27] Kishor Kunal et al. “GANA: Graph Convolutional Network Based Automated Netlist Annotation for Analog Circuits.” In: *DATE*. 2020, pp. 55–60.
- [28] Inga Abel, Maximilian Neuner, and Helmut Graeb. “A functional block decomposition method for automatic op-amp design”. In: *Integration* 85 (2022), pp. 108–120.
- [29] *ACST: Analog Circuit Synthesis Tool*. 2026. URL: <https://github.com/inga000/acst> (visited on 01/30/2026).
- [30] Michael Eick and Helmut E Graeb. “MARS: Matching-driven analog sizing”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 31.8 (2012), pp. 1145–1158.
- [31] Pallets. *Flask Documentation (3.1.x)*. 2026. URL: <https://flask.palletsprojects.com/en/stable/> (visited on 04/06/2026).
- [32] *schengen: Netlist to Schematic Conversion via Constraint Generation*. 2026. URL: <https://anonymous.4open.science/r/schengen/> (visited on 04/22/2026).